**TRIBHUVAN UNIVERSITY**
**INSTITUTE OF ENGINEERING**
**HIMALAYA COLLEGE OF ENGINEERING**
**[CODE: CT-755]**

A
PROGRESS REPORT
ON

# CIVIC COMPLAINT CLASSIFICATION SYSTEM USING DISTILBERT


**SUBMITTED BY:**
ASHMITA THAPA (HCE078BCT008)
JEEVIKA SUBEDI (HCE078BCT016)
NAYANA BHATTA (HCE078BCT024)
UNISHA CHAULAGAIN (HCE078BCT045)


A PROGRESS REPORT SUBMITTED TO DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR BACHELORS DEGREE IN ELECTRONICS, COMMUNICATION AND INFORMATION ENGINEERING


DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING
LALITPUR, NEPAL


**AUGUST, 2025**

# (CIVIC COMPLAINT CLASSIFICATION SYSTEM USING DISTILBERT)

**BY**

Ashmita Thapa (HCE078BCT008)
Jeevika Subedi(HCE078BCT016)
Nayana Bhatta (HCE078BCT024)
Unisha Chaulagain (HCE078BCT045)

A progress report submitted in partial fulfillment of the requirements for the **Bachelor's Degree in Electronics, Communication and Information Engineering**

Department of Electronics and Computer Engineering
**HIMALAYA COLLEGE OF ENGINEERING**
Tribhuvan University
Lalitpur, Nepal

**August, 2025**

# Acknowledgment

I would like to express my sincere gratitude to all those who have supported and guided me throughout the development of this project.First and foremost, I am deeply thankful to our Vice Principal, **Er. Ashok Gharti Magar**, for his invaluable encouragement and continuous support.

I am also grateful to our Head of Department, **Er. Devendra Kathayat**, and Deputy Head of Department, **Er. Ramesh Tamang**, for their expert guidance and constructive feedback during the project.

Special thanks to the Project Coordinator, **Er. Narayan Chettri**, whose coordination and assistance greatly contributed to the successful progress of this work.

Their encouragement, advice, and support have been instrumental in completing this project, and I am truly appreciative of their contributions.

# Table of Contents

**Acknowledgement** **iii**

**List of Figures** **vi**

**List of Abbreviations** **vii**

**1 INTRODUCTION** **1**

1.1 Background . . . . . 1

1.2 Problem statements . . . . . 1

1.3 Objective . . . . . 1

1.4 Scope and Applications . . . . . 1

**2 LITERATURE REVIEW** **3**

2.1 Related work . . . . . 3

2.2 Related Theory . . . . . 4

**3 REQUIREMENT ANALYSIS AND SYSTEM DESIGN** **5**

3.1 Requirement Analysis . . . . . 5

3.1.1 Functional Requirements . . . . . 5

3.1.2 Non-Functional Requirements . . . . . 5

3.2 Feasibility Analysis . . . . . 6

3.3 System Design . . . . . 7

3.3.1 System Architecture . . . . . 7

3.3.2 UseCase Diagram . . . . . 8

3.3.3 ER Diagram . . . . . 9

3.3.4 Data Flow Diagram . . . . . 10

3.3.5 Sequence Diagram . . . . . 12

3.3.6 Activity Diagram . . . . . 13

**4 Methodology** **15**

4.1 System Development Model . . . . . 15

4.2 System Workflow Based on Block Diagram . . . . . 15

4.2.1 User Input Layer (Text, Voice, GPS) . . . . . 15

4.2.2 Preprocessing Module . . . . . 16

4.2.3 AI-Based Complaint Classification Module . . . . . 17
4.2.4 Backend System (Django) . . . . . 20
4.2.5 Notification System . . . . . 21
4.3 Security and Deployment Strategy . . . . . 21

**5 Work Progress** **23**

**6 Results and Analysis** **24**
6.1 Class-wise Performance Evaluation . . . . . 24
6.2 Confusion Matrix Analysis . . . . . 24
6.3 Prediction Confidence Analysis . . . . . 25
6.4 Overall Model Performance . . . . . 26
References . . . . . 26

# List of Figures


3.1 System Architecture . . . . . 7
3.2 UseCase . . . . . 8
3.3 ER Diagram . . . . . 9
3.4 DFD Level 0 . . . . . 10
3.5 DFD Level 1 . . . . . 11
3.6 Sequence Diagram . . . . . 12
3.7 Activity Diagram . . . . . 13

4.1 System Block Diagram . . . . . 15
4.2 AI-Based Complaint Classification Module . . . . . 17

6.1 Class-wise metrics . . . . . 24
6.2 Confusion matrix . . . . . 25
6.3 Predicted probabilitiy distibution per class . . . . . 26

# List of Abbreviations

| | |
|---|---|
| **AI** | Artificial Intelligence |
| **DRF** | Django REST Framework |
| **FCM** | Firebase Cloud Messaging |
| **NLP** | Natural Language Processing |
| **API** | Application Programming Interface |
| **GPS** | Global Positioning System |
| **ORM** | Object-Relational Mapping |
| **CI/CD** | Continuous Integration / Continuous Deployment |
| **HTML** | HyperText Markup Language |
| **SQL** | Structured Query Language |
| **PAN** | Permanent Account Number |

# 1. INTRODUCTION

## 1.1 Background

In Nepal, many citizen especially in rural and underserved areas face challenges in reporting civic issues such as water shortages, infrastructure damage, or public safety concerns. Limited digital literacy, unclear departmental procedures, and language barriers often prevent timely and effective communication with authorities.

**Civic Complaint Classification system** aims to address these challenges by providing a smart, AI-powered mobile application that allows users to report complaints using voice and text in local languages like Nepali or English. The system uses Artificial Intelligence (AI) and Natural Language Processing (NLP) to interpret and classify complaints, and automatically routes them to the appropriate government authority, such as the municipality or police department. This ensures that citizens can raise their voices easily without needing to navigate complex structures.

## 1.2 Problem statements

Despite increasing access to mobile technology, the current civic complaint systems in Nepal are not effectively serving the broader population. Major problems include:

- Citizens often don't know which authority to contact for specific issues.
- Language barriers and low literacy levels limit access to digital services.
- Manual systems lead to delays, misrouted complaints, and lack of accountability.
- There is no centralized platform that connects citizens and government bodies efficiently.

## 1.3 Objective

To develop a mobile application that allows users to submit civic complaints via text or voice, and classify and route them to the appropriate authority using distilbert.

## 1.4 Scope and Applications

- Enables citizens to submit complaints using voice and text in local languages, including Nepali.

- Uses AI and NLP to automatically analyze, categorize, and forward complaints to the correct authority.
- Designed for rural and low-literacy users, ensuring accessibility for all, regardless of digital skills.
- Ensures real-time routing and acknowledgment of complaints to improve response time and accountability.
- Offers a centralized dashboard for authorities to track, manage, and respond to complaints efficiently.
- Supports integration with government systems, making it scalable and adaptable for national-level deployment.

# 2. LITERATURE REVIEW

## 2.1 Related work

Civic complaint systems provide citizens with the means to report local issues such as road damage or water leakage. India's Centralized Public Grievance Redress and Monitoring System (CPGRAMS) is an example of a large-scale platform, though it primarily relies on manual input and lacks AI capabilities [1]. Recent advances have proposed mobile applications integrating geotagging and SMS alerts, emphasizing the need for location-aware complaint management to enable efficient issue tracking and resolution [2].

NLP is critical for understanding and categorizing text or voice-based complaints. Earlier approaches utilized algorithms such as Naïve Bayes and Support Vector Machines for text classification in civic applications [3]. More recently, transformer-based models like BERT have substantially improved the contextual understanding of complaints, especially in multilingual contexts [4]. Specifically, pretrained multilingual BERT models have shown strong performance for Nepali and Hindi text classification [5].

Image classification using Convolutional Neural Networks (CNNs) enables automatic identification of civic issues such as potholes or broken streetlights from user-submitted photos. Research has shown that CNNs can achieve high accuracy in detecting road surface conditions, providing critical support to complaint systems where textual descriptions may be insufficient [6].

Geolocation technologies provide spatial context to complaints, allowing authorities to visualize and prioritize issues geographically. Open-source tools like Leaflet.js have been successfully implemented for real-time complaint mapping and visualization, facilitating improved municipal response in resource-constrained settings [7].

To improve accessibility, modern complaint systems incorporate multimodal input methods including text, voice, and images. Integrating speech recognition systems in mobile apps has enhanced usability for users with literacy or disability challenges, ensuring more inclusive civic participation [8].

Real-time notification systems, such as Firebase Cloud Messaging, are essential to keep citizens informed about complaint status and actions taken by authorities. Research indicates that such feedback mechanisms significantly improve trust and engagement between citizens and municipal bodies [9].

## 2.2 Related Theory

Civic engagement in urban governance has increasingly embraced digital platforms to improve public service delivery. However, developing countries such as Nepal continue to face challenges in efficiently processing civic complaints due to limitations in infrastructure and technology integration. The use of Artificial Intelligence (AI) techniques, particularly Natural Language Processing (NLP), image classification, and geolocation mapping, has demonstrated considerable potential to enhance transparency, citizen participation, and responsiveness in complaint management systems [1].

NLP techniques enable machines to understand human language, which is critical for analyzing text and voice complaints. Classical algorithms like Naïve Bayes and Support Vector Machines (SVM) have been used traditionally for text classification tasks [3]. The emergence of transformer-based architectures such as BERT has significantly advanced language understanding capabilities, especially in multilingual and context-rich environments [4]. Pretrained multilingual BERT models effectively handle languages such as Nepali and Hindi, improving classification accuracy [5].

Computer Vision, particularly through Convolutional Neural Networks (CNNs), plays an important role in image-based civic issue detection. CNNs automatically extract features from images, enabling classification of issues like potholes and damaged infrastructure with high precision [6].

Geolocation mapping provides vital spatial context for complaints, aiding authorities in visualization and prioritization. Open-source mapping libraries such as Leaflet.js facilitate dynamic, real-time complaint tracking even in areas with limited digital infrastructure [7].

Speech recognition systems improve accessibility by allowing voice input, thereby accommodating users with varying literacy and physical abilities. Integration of speech-to-text APIs enhances inclusiveness and expands user participation [8].

Real-time notification and feedback mechanisms maintain communication loops between citizens and authorities, improving transparency and fostering trust in public service delivery [9].

# 3. REQUIREMENT ANALYSIS AND SYSTEM DESIGN

## 3.1 Requirement Analysis

### 3.1.1 Functional Requirements

- **Complaint Submission:** Allow users to submit complaints via voice recordings, text input.
- **Voice-to-Text Conversion:** Automatically convert voice recordings to text and analyze the content using AI for classification.
- **Geo-location Tagging:** Automatically add the location where the complaint was made to help authorities find and fix the problem faster.
- **Status Updates:** Provide feedback or status updates to users regarding their complaints.
- **Complaint Categorization:** Categorize complaints into relevant departments such as water supply, sanitation, or electricity.
- **Complaint Routing:** Route complaints automatically to the appropriate government authority and provide acknowledgment to users.
- **Complaint Management:** Enable authorities to view, update, and manage complaints through a centralized dashboard.

### 3.1.2 Non-Functional Requirements

- **Usability:** Simple, intuitive interface accessible to users with limited digital literacy.
- **Performance:** Real-time complaint processing and routing with minimal delays.
- **Reliability:** High availability of the system with regular backups and failover support.
- **Security:** Data encryption, secure user authentication, and privacy compliance to protect sensitive information.

- **Scalability:** Ability to handle growing numbers of users and complaints as adoption increases.

## 3.2 Feasibility Analysis

### 3.2.1 Technical Feasibility

The Civic Complaint System is technically feasible as it is built using widely adopted and reliable technologies. The mobile app will be developed using Flutter, which supports voice and text, inputs. On the backend, Django and Django REST Framework provide a stable infrastructure for API handling, complaint logic, and data management. The use of well-supported AI tools like Scikit-learn, ensures accurate NLP . Integration with APIs such as mozzila Deepspeech for speech to text detection and NLTK for text cleaning and translation , and Leaflet.js further strengthens the technical foundation. Since all tools are well-documented and have active community support, the technical risk is minimal.

### 3.2.2 Operational Feasibility

The project is highly feasible from an operational perspective. It is designed with inclusivity in mind, allowing users to submit complaints using voice in local languages, which reduces barriers for those with low literacy. The interface is simple and user-friendly, requiring minimal digital skills. On the government's side, the platform provides categorized, location-tagged complaints via a centralized dashboard, simplifying case management. Minimal training is required for municipal workers, and the process can be easily incorporated into daily administrative operations.

### 3.2.3 Economic Feasibility

Civic Complaint is economically viable. Most of the technologies used are open-source or offer generous free tiers, reducing initial development costs. Hosting services like Render or Railway are affordable for startups and civic projects. Long-term costs are also manageable due to the system's cloud-based architecture and modular structure. The impact in terms of improved citizen engagement, faster problem resolution, and greater government accountability provides strong value for money.

### 3.2.4 Schedule Feasibility

The project follows an incremental and iterative development approach, which makes the schedule both flexible and achievable. By building a Minimum Viable Product (MVP) first—focusing on core features like voice complaint submission, AI classification, and dashboard integration—functional results can be delivered within a few months. Later phases can introduce advanced features such as real-time analytics, translation, and offline

support. With clear goals, task breakdowns, and a committed development team, the project timeline is realistic and can be met without major delays.

## 3.3 System Design

### 3.3.1 System Architecture

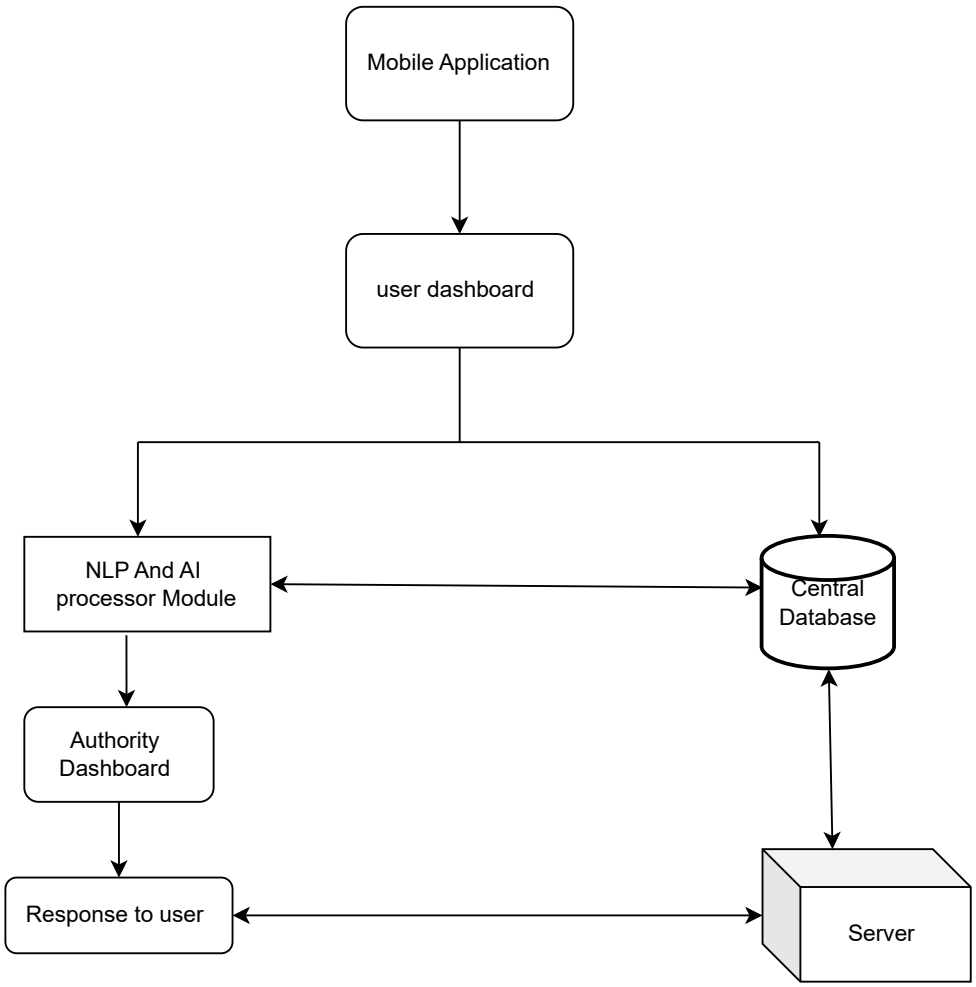


Figure 3.1: System Architecture

The system architecture diagram includes several key components that work together to manage civic complaints. It starts with the User, who submits a complaint through the User Interface, which is a mobile application. The submitted complaint is processed by the NLP and AI Processor Module, where it is analyzed and classified using natural language processing techniques. The classified complaint is then passed to the Complaint Router Module, which identifies the correct authority for handling the issue based on the complaint type. All of this communication and logic is managed by the Server, which

acts as the central controller. The data is stored in the Central Database, which keeps records of users, complaints, and statuses. Once an authority takes action, the Response Handler Module updates the system with their response. Finally, the Notification System sends real-time updates to both the user and the authority to keep them informed about the complaint status.

### 3.3.2 UseCase Diagram

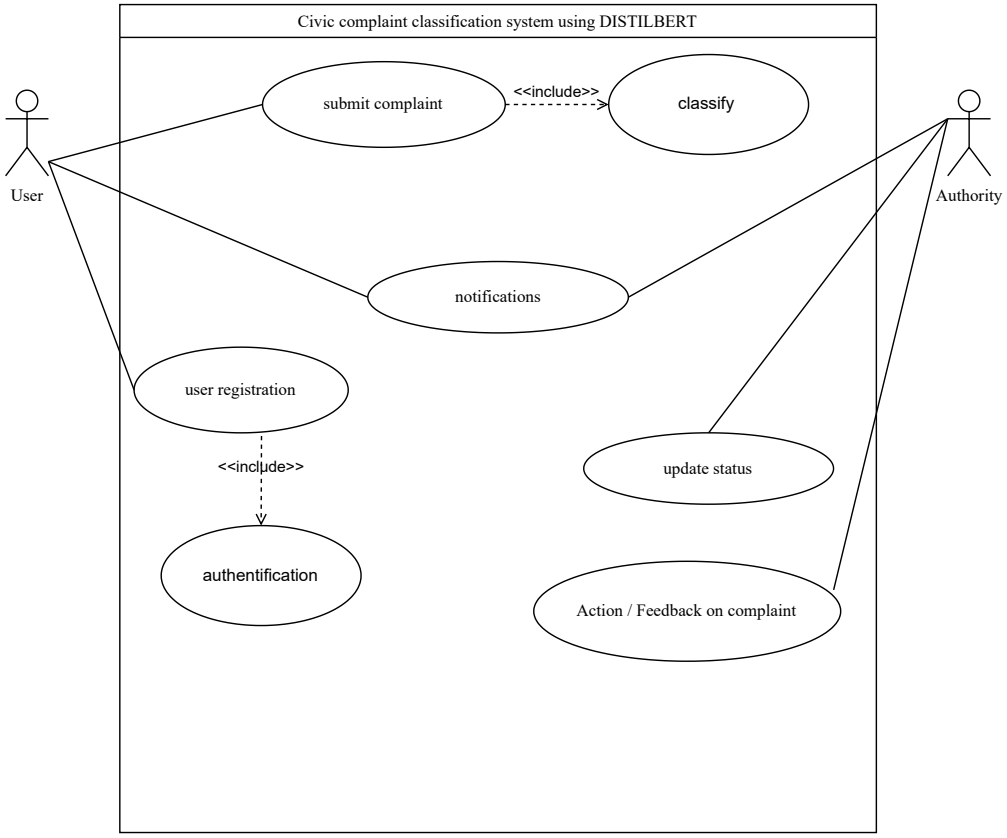


Figure 3.2: UseCase

This use case diagram illustrates the key interactions within the Civic Complaint Classification System using DistilBERT. The central actor is the User, who can register, submit complaints, receive notifications, and view actions or feedback on their complaints. Once a complaint is submitted, it is passed through the classify process, which uses the DistilBERT model to automatically categorize it (electricity, water, road, garbage). On the other side, the Authority (such as municipal staff) interacts with the system to update the complaint status after resolving or reviewing it.

### 3.3.3 ER Diagram

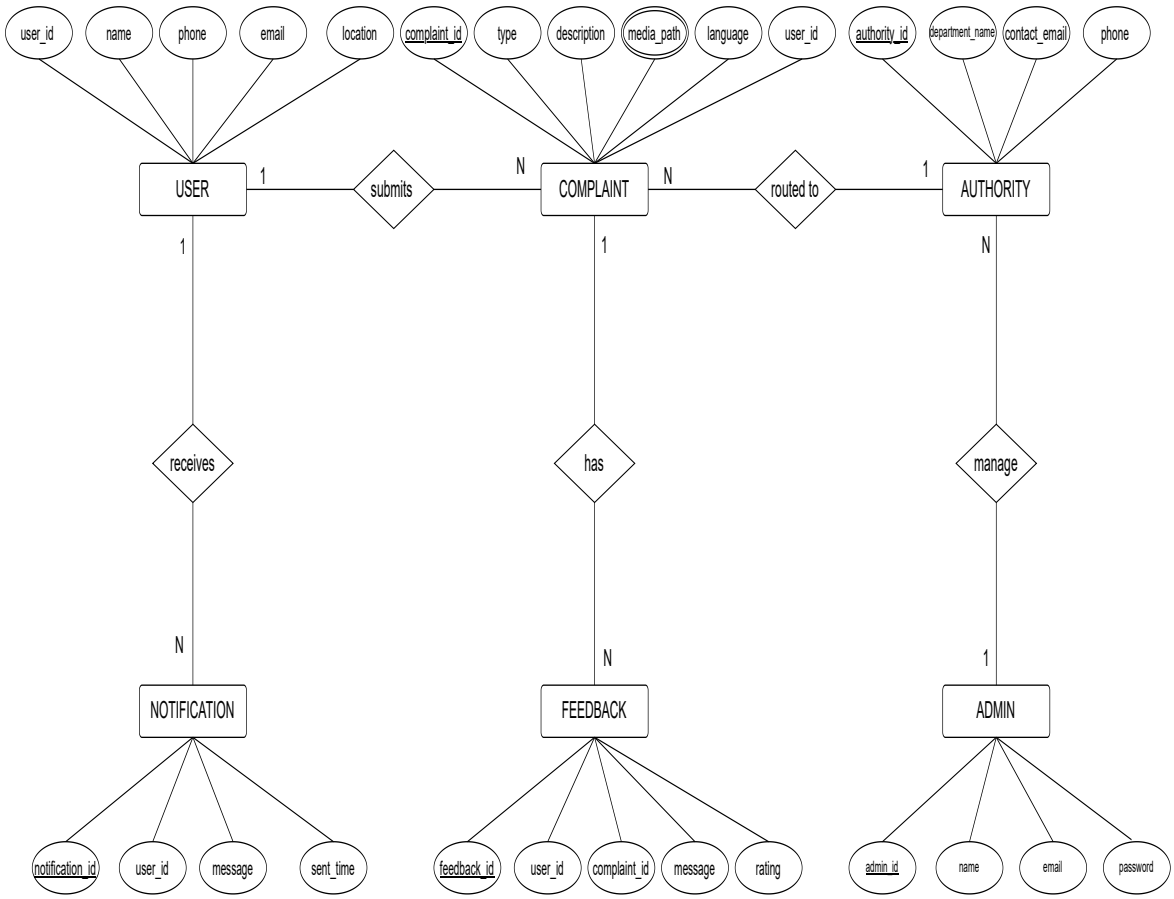


Figure 3.3: ER Diagram

The Entity Relationship (ER) diagram for Civic Complaint defines the data structure and relationships among the core components of the system. At the center is the User entity, which captures essential information such as name, contact details, and location. Each user can submit multiple Complaints, which include fields like complaint type, description, status, timestamp, and media attachments. Complaints are automatically routed to the relevant Authority, which is managed by an Admin. Users may provide Feedback on a complaint after it's processed, linking user experience directly to the complaint resolution. Additionally, the system sends Notifications to users to keep them informed about complaint updates and responses. These relationships and attributes ensure the system is well-structured, scalable, and supports seamless interaction between users and municipal bodies.

### 3.3.4 Data Flow Diagram

#### a. DFD LEVEL 0

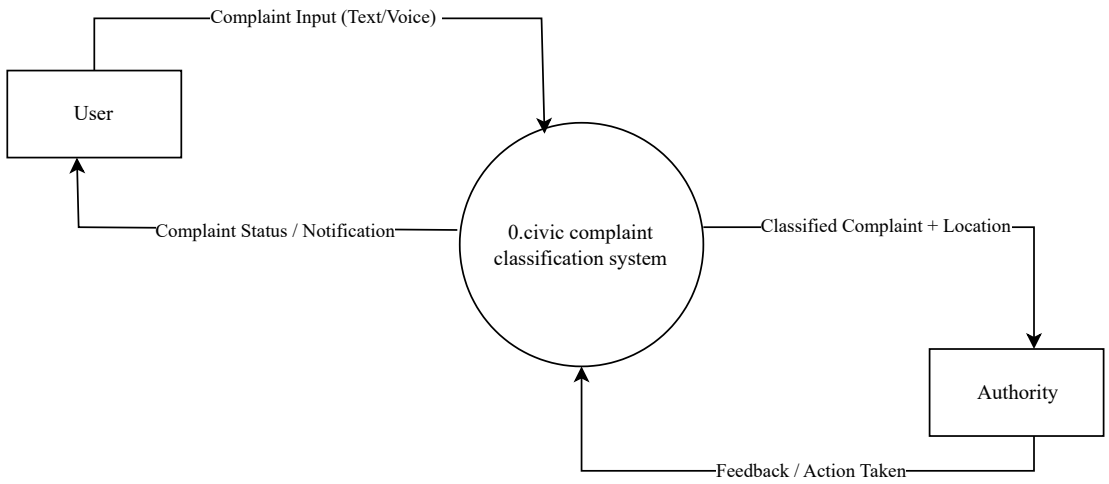


Figure 3.4: DFD Level 0

In the Level 0 Data Flow Diagram (DFD) of Civic Complaint, the primary external entities are the User and the Authority. The process begins when a user submits a complaint to the system using text,voice. The Civic complaint system receives this data, processes it, and stores it in the internal database. Based on the complaint type and location, the system forwards the information to the appropriate authority for further action. Once the authority reviews and responds to the complaint, the system generates a real-time notification to update the citizen on the complaint status.

## b. DFD LEVEL 1

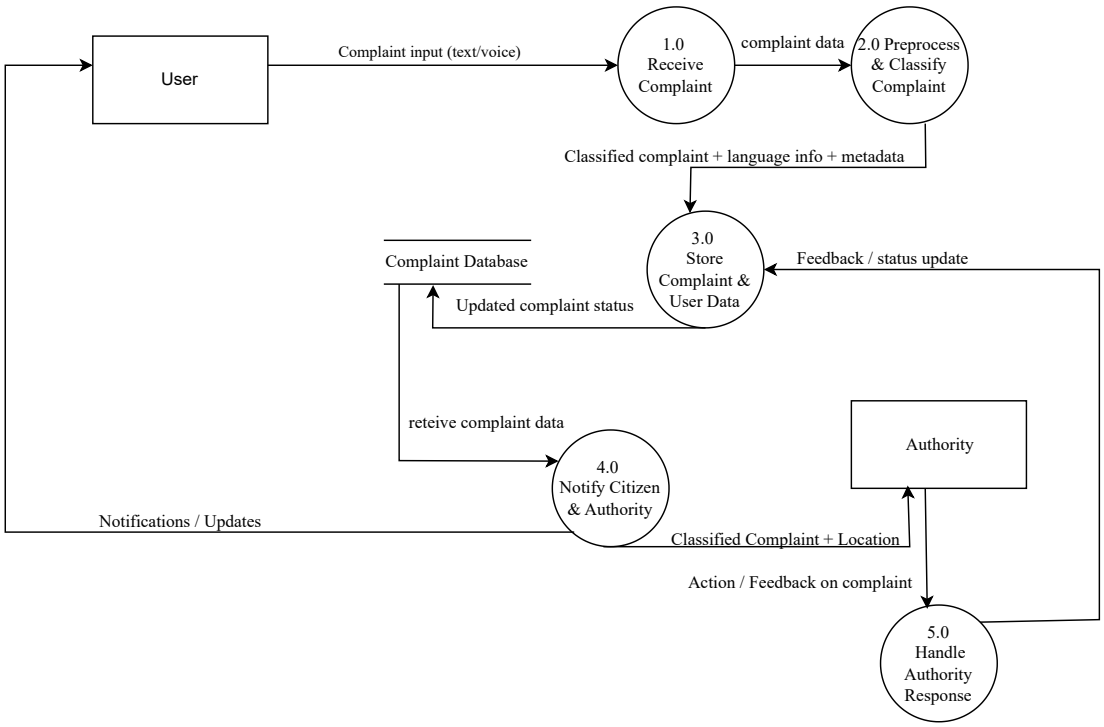


Figure 3.5: DFD Level 1

The Level 1 DFD breaks down the system into detailed processes. Users submit complaints using text, voice. The system validates and classifies these complaints using AI techniques like NLP. It then maps the complaint's location to the relevant municipal area. The complaint is forwarded to the appropriate municipal department for action. Once addressed, the system sends real-time feedback and notifications back to the citizen, ensuring transparent communication and timely resolution.

### 3.3.5 Sequence Diagram

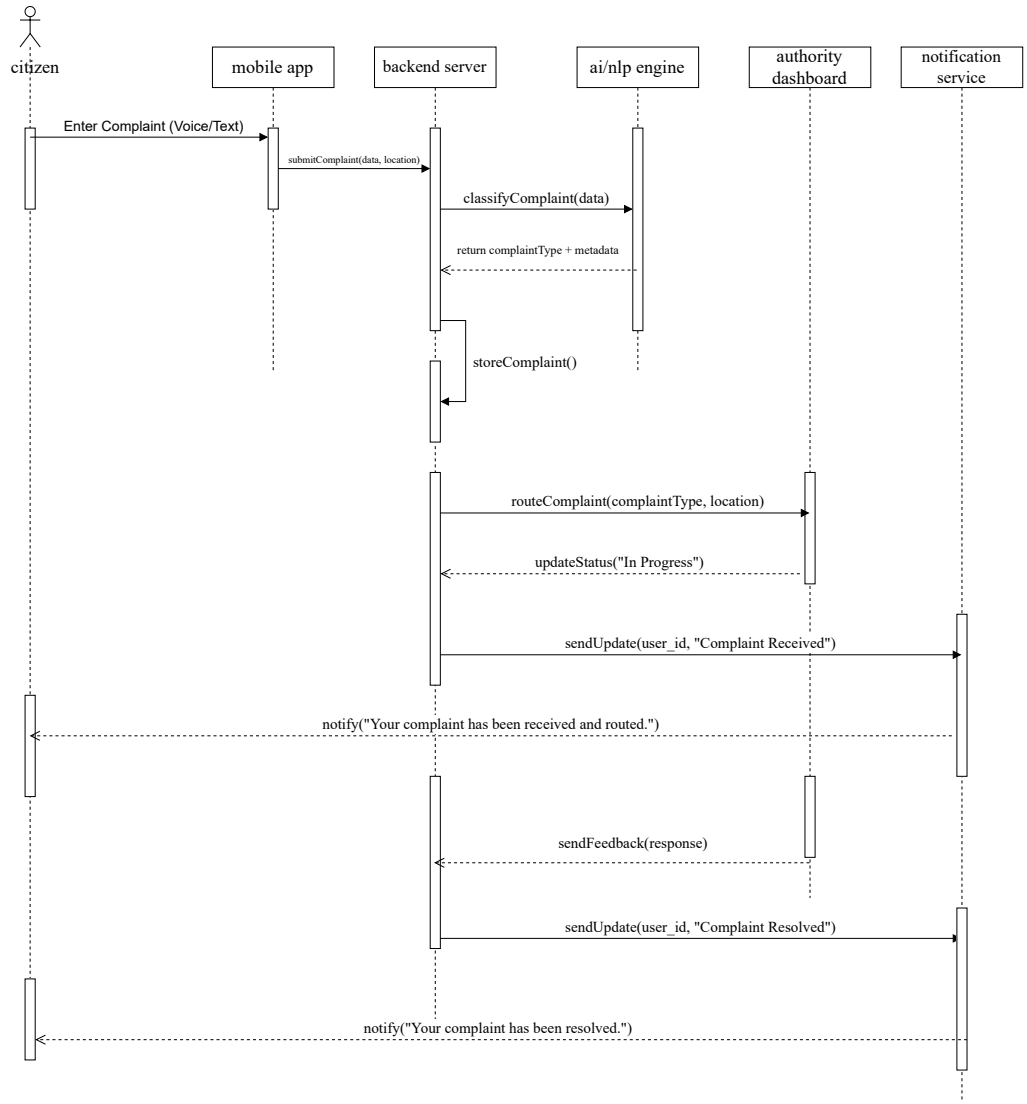


Figure 3.6: Sequence Diagram

The sequence diagram for Civic Complaint illustrates the flow of interactions involved in the civic complaint submission and response process. It begins with the citizen launching the mobile application and submitting a complaint using voice and text input. The app forwards the complaint data, along with GPS location, to the backend server. The server then interacts with the AI/NLP engine to classify the complaint based on its content and metadata. Once classified, the complaint is stored in the system and automatically routed to the appropriate authority via the authority dashboard. The authority reviews and updates the complaint status, which is then communicated back to the citizen through the notification service using push notifications or email. The diagram clearly outlines the

real-time and interactive nature of the system, ensuring quick and transparent communication between citizens and government bodies.

### 3.3.6 Activity Diagram

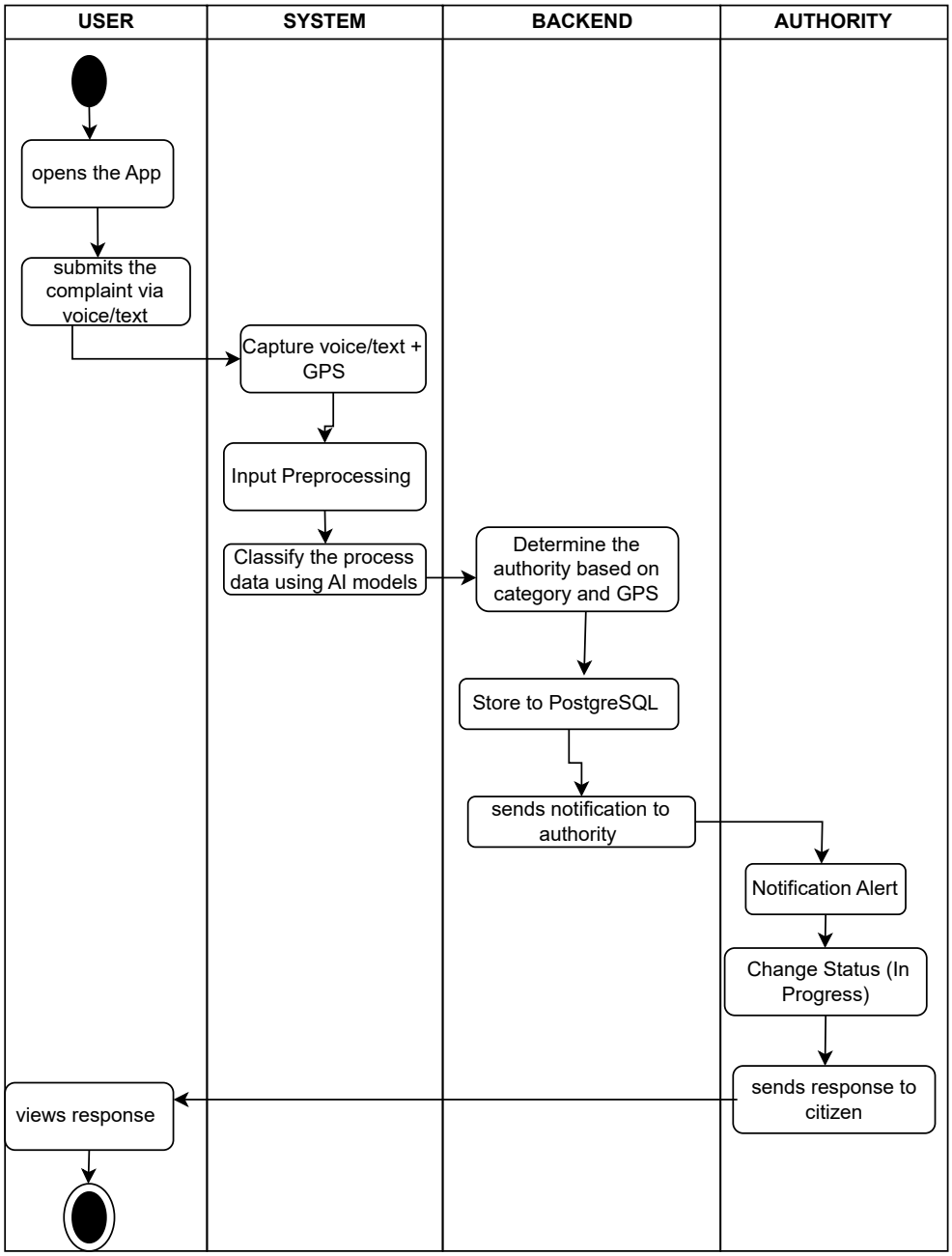


Figure 3.7: Activity Diagram

The activity diagram illustrates the end-to-end workflow of the Civic complaint system across four main entities: User, System (App + AI), Backend, and Authority. A user initiates the process by submitting a civic complaint through a mobile app via text or voice, along with automatic GPS capture. The system validates and preprocesses the input, performs language detection, translation, and text cleaning, then classifies the complaint using AI models like TF-IDF, SVM, or DistilBERT. The classified complaint is sent to the backend, where Django handles routing based on location and category, stores data in PostgreSQL, and triggers status tracking. Finally, relevant municipal authorities are notified in real-time and take action, with updates sent back to the user through notifications. The process ensures efficient, AI-driven complaint handling and resolution.

# 4. Methodology

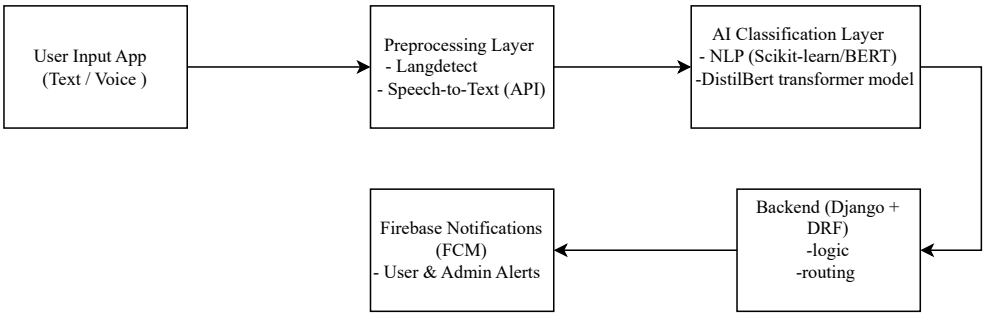


Figure 4.1: System Block Diagram

The methodology adopted for this project outlines a systematic design, development, and deployment process of **Civic Complaint classification system using DISTILBERT**. This intelligent civic complaint management system allows citizens to submit voice and text-based complaints using a mobile interface. An advanced AI engine powered by a fine-tuned DistilBERT model classifies these complaints and routes them to the appropriate municipal authority, with complete backend and database support. The methodology follows a modular, iterative development model focusing on open-source tools, scalability, and usability.

## 4.1 System Development Model

An iterative, modular development approach is followed:

- Each system component (input, preprocessing, AI classification, backend, and notification) is developed and tested independently.
- Agile feedback and evaluation loops refine accuracy at every phase.
- The entire pipeline is connected using RESTful APIs and database integration.

## 4.2 System Workflow Based on Block Diagram

### 4.2.1 User Input Layer (Text, Voice, GPS)

**Tools:** Flutter (app), Android/iOS microphone and GPS module.

**Description:** Users submit complaints via a mobile application built using Flutter. Two types of input are supported:

- **Text input:** Users can type complaints in English or Nepali.
- **Voice input:** Audio is captured via the mobile microphone and transcribed to text.

**GPS Integration:** User location is auto-captured using the mobile's GPS.
**Tools Used:**

- Flutter (UI)
- Device microphone + GPS modules

**Output:** Raw text/audio complaint + geolocation.

### 4.2.2 Preprocessing Module

This layer ensures input standardization before AI classification. It converts raw voice or text into clean English text ready for classification.

**Steps and Algorithms:**

- **Language Detection:** Using `Langdetect` to identify input language.
- **Speech-to-Text:** Handled using **Mozilla DeepSpeech** (offline ASR).
- **Translation:** Nepali text is translated into English using custom rule-based translation.
- **Text Cleaning:** Performed using `NLTK` – lowercasing, tokenization, stopword removal, and stemming.

**Output:** Cleaned English-language complaint.
**Libraries Used:** NLTK, DeepSpeech, Langdetect.

### 4.2.3 AI-Based Complaint Classification Module

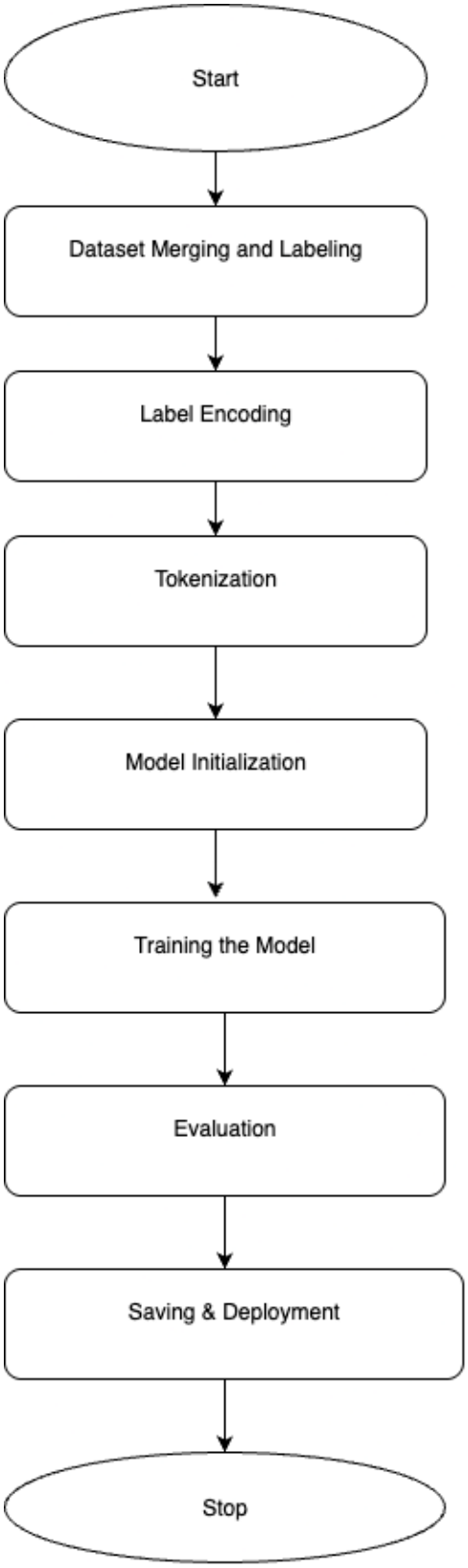


Figure 4.2: AI-Based Complaint Classification Module

The **AI-Based Complaint Classification Module** is the intelligent engine that powers the Civic Complaint system. It automatically categorizes user-submitted complaints into four predefined civic categories: *electricity*, *water*, *road*, or *garbage*. This classification is crucial as it ensures that the complaint is routed to the right municipal department without manual intervention. To achieve this, we fine-tuned a pre-trained transformer-based model, **DistilBERT**, which offers high performance with faster execution and fewer parameters. The complete development of this module involved several systematic steps: dataset preparation, label encoding, tokenization, model training, evaluation, and final deployment.

## 1. Dataset Merging and Labeling

To begin, we collected complaint datasets from various sources. These datasets contained textual complaints categorized under electricity, water, road, or garbage. However, the structure of each dataset varied—some used keys like `"text"`, while others used `"message"` or `"complaint"`. To unify the format, we created a Python script named `merge.py`, which automatically traversed each dataset file, extracted complaint text from any known key, and assigned the correct category label. All cleaned and labeled data were compiled into a single `pandas` DataFrame. We removed any missing or empty entries using the `dropna()` function to ensure data quality. The final dataset was saved as `merged_complaints.csv` and consisted of two main columns: `text` (the actual complaint) and `label` (the complaint category). This dataset served as the foundation for model training.

## 2. Label Encoding

Since machine learning models require numerical inputs, we converted the string labels (`electricity`, `water`, etc.) into integer values using `LabelEncoder` from `scikit-learn`. Each category was mapped to a unique integer: electricity $\rightarrow$ 0, water $\rightarrow$ 1, road $\rightarrow$ 2, garbage $\rightarrow$ 3. This transformation allowed the model to interpret the categories as classification targets. The label encoding follows the formula:

$$y_{\text{encoded}} = \text{index}(y_{\text{original}})$$

which means the encoded label is simply the index of the original label in a predefined list of classes. This step is essential because deep learning models like DistilBERT can only process numerical outputs for classification tasks.

## 3. Tokenization Using Transformers

Once the labels were encoded, the next step was to convert the raw complaint texts into a format the model could understand. We used the `DistilBertTokenizerFast` from

Hugging Face to perform tokenization. Tokenization breaks down each sentence into smaller parts called tokens and converts them into numeric IDs (`input_ids`). It also generates an `attention_mask` which tells the model which parts of the input are actual text and which are just padding (used to ensure equal input lengths). Additionally, special tokens like `[CLS]` (start of sequence) and `[SEP]` (end of sequence) are added automatically. Each token is then mapped to a 768-dimensional vector, known as an embedding:

$$E(w_i) \in R^{768}$$

This process allows the model to understand the meaning of each word in context, enabling accurate classification.

## 4. Model Initialization – DistilBERT

We chose **DistilBERT** for this task due to its efficient architecture that retains most of BERT's accuracy while being smaller and faster. Using the `DistilBert For Sequence Classification` class, we initialized a model with `num_labels=4` for our four complaint categories. Internally, DistilBERT uses a transformer encoder with self-attention mechanisms that help the model understand the relationships between words in a sentence. The attention mechanism is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where $Q$, $K$, and $V$ are the query, key, and value matrices, and $d_k$ is the dimension of the key vectors. This formula helps the model focus more on important words when making predictions. Additionally, since transformers are not sequential models, we used positional encodings to preserve word order:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

These mathematical functions assign each position a unique representation, enabling the model to understand sentence structure.

## 5. Training Configuration

To train the model, we used Hugging Face's `Trainer` API, which simplifies training and evaluation. We configured the model with the following hyperparameters: 3 epochs, batch size of 8, a learning rate of 5e-5, and the AdamW optimizer. The model was trained using the categorical cross-entropy loss function, which is appropriate for multi-class classification problems. The formula for this loss is:

$$\mathcal{L} = -\sum_{i=1}^{C} y_i \cdot \log(\hat{y}_i)$$

Here, $y_i$ is the true one-hot encoded label, $\hat{y}_i$ is the predicted probability for each class, and $C$ is the number of categories (4 in our case). This loss function penalizes incorrect predictions and helps the model improve during training.

### 6. Evaluation and Metrics

After training, the model was evaluated using several key metrics: **accuracy**, **precision**, **recall**, and **F1-score**. Accuracy measures the overall correctness of the model. Precision indicates how many of the complaints classified into a category were actually correct. Recall measures how well the model found all actual instances of a class. F1-score provides a balance between precision and recall, especially useful when class distribution is uneven. The F1-score formula is:

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

This metric ensures that the model does not perform well on one metric at the expense of the other. A confusion matrix was also used to visualize correct vs. incorrect predictions for each class, helping us understand model performance in detail.

### 7. Saving and Deployment

Finally, the trained model was saved using Hugging Face's `.save_pretrained()` function and integrated into the backend of the CivicConnect application. When a user submits a complaint via the app, the system tokenizes the input, feeds it to the model, and immediately receives the predicted category. This category is then used to forward the complaint to the appropriate department. This deployment allows for real-time, automated complaint classification, eliminating the need for manual sorting and significantly speeding up response times.

### 4.2.4 Backend System (Django)

**Purpose:** Handle the core business logic, from receiving complaints to routing and updates.

**Implementation Details:**

- **API Handling:** Django REST Framework exposes secured REST APIs.
- **Routing Logic:**
  - Based on the type of complaint and GPS, the complaints are forwarded to the relevant municipal department or ward office.

  - Example: Complaint about "Garbage" in "Ward 2" → routed to sanitation officer of Ward 2.

- **Authentication:** Token-based authentication using DRF.
- **Status Tracking:** Complaint status (Open, In Progress, Resolved) updated via backend triggers.
- **Integration:** Geolocation Navigator is used for mapping geolocated complaints in the dashboard.
- **Database:**
  - SQLite is used as the primary backend database (default in Django).
  - Stores all user data, complaint records, authentication tokens, and status updates.
  - Lightweight and suitable for development and small-scale deployment.

**Output:** Complaint routed and stored, status updates managed.

### 4.2.5 Notification System

**Function:** Alert both users and municipality staff in real time.

- **Firebase Cloud Messaging (FCM):**
  - Sends complaint confirmation to user.
  - Sends new task alerts to municipal officers based on category and location.
- **Routing Example:**
  - Complaint classified as `Water Problem`, GPS = "Ward 7, Lalitpur" → Ward 7 water department notified.

**Output:** Real-time updates to stakeholders.

## 4.3 Security and Deployment Strategy

- All REST APIs are secured with token-based authentication.
- Data is stored with encryption and backup enabled IN Django sqlite.
- The deployment is done using platforms like **Render** or **Railway**, with GitHub Actions for CI/CD.

- Only authorized users and verified municipal personnel can access the dashboard and complaint data.

This methodology supports a real-time AI-driven civic grievance system using free and open source tools. Each phase - from multilingual voice / text input to NLP classification, routing, and notification - is treated as a modular unit. The use of AI models such as DistilBERT and classical classifiers ensures accurate complaint recognition, while Database and Django guarantee secure and efficient complaint management. The system is scalable, language sensitive and local sensitive, empowering citizens with an accessible platform to raise their concerns.

# 5. Work Progress

## Work Completed

- Civic complaint datasets collected and analyzed.
- AI-based complaint classification model trained using NLP techniques (DistilBERT).
- Backend APIs developed using Django REST framework.
- Designing and developing the frontend (mobile app).
- User authentication and role-based dashboards.

## Work Remaining

- Integrating voice complaint features.
- Real-time geolocation tracker.
- Basic structure of the backend set up and integrated with the trained model.

# 6. Results and Analysis

This section presents the performance analysis of the proposed AI-based civic complaint classification system. Various graphical representations are used to clearly demonstrate the effectiveness, accuracy, and reliability of the trained DistilBERT model.

## 6.1 Class-wise Performance Evaluation

Figure 6.1 illustrates the class-wise precision, recall, and F1-score for each complaint category. Precision measures how accurately the model predicts a particular class, recall indicates how well the model identifies all relevant complaints, and the F1-score provides a balanced measure of both. The graph shows that all complaint categories achieve scores close to 1.0, indicating that the model performs exceptionally well across all classes. This confirms that the system is capable of handling different types of civic complaints without favoring or neglecting any specific category.

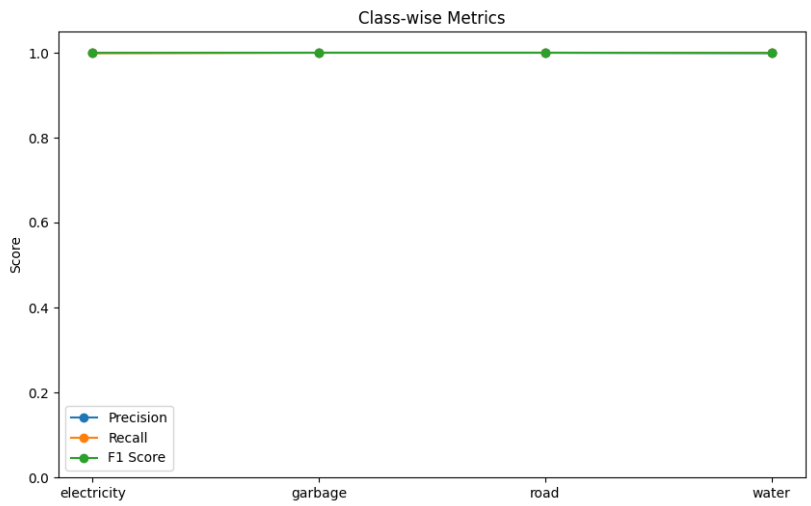


Figure 6.1: Class-wise metrics

## 6.2 Confusion Matrix Analysis

Figure 6.2 presents the confusion matrix of the trained model. The matrix compares the actual complaint categories with the predicted categories generated by the system. Most of the values are concentrated along the diagonal, which indicates that the model correctly classifies the majority of complaints. The minimal presence of off-diagonal values

shows that misclassifications are very rare. This demonstrates the strong discriminatory capability of the model in distinguishing between different types of civic complaints.

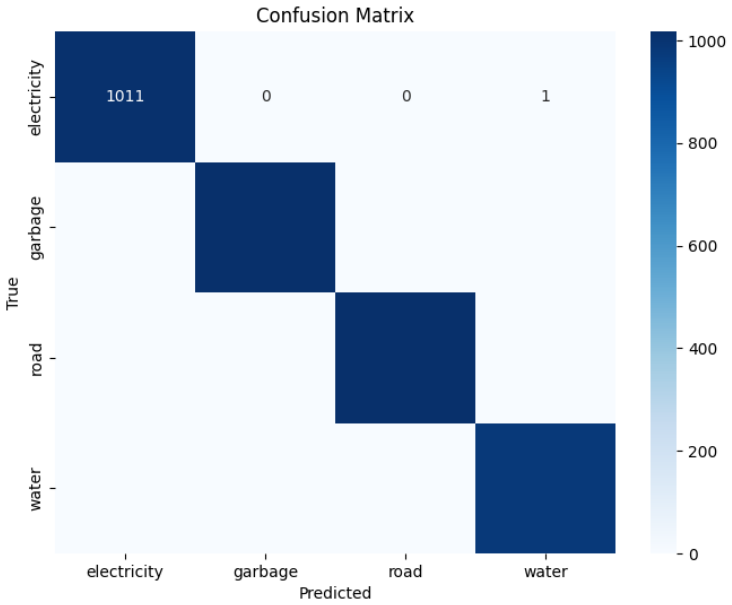


Figure 6.2: Confusion matrix

## 6.3 Prediction Confidence Analysis

Figure 6.3 illustrates the predicted probability distribution for each class. This graph represents how confident the model is while making predictions. The results show that most predictions have probability values close to 1.0, indicating a high level of confidence. Such strong confidence suggests that the model has learned meaningful patterns from the data and is reliable for real-world deployment, where accurate and confident predictions are crucial for decision-making.

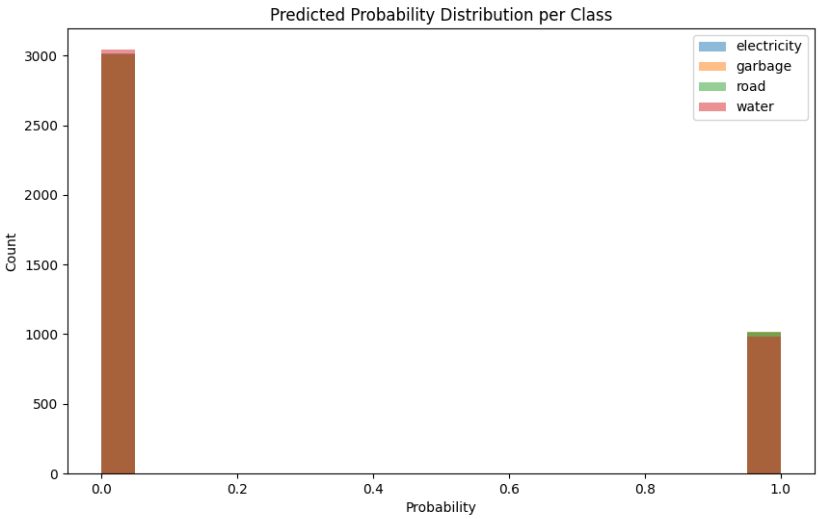


Figure 6.3: Predicted probabilitiy distibution per class

## 6.4 Overall Model Performance

Based on the evaluation results, the proposed DistilBERT-based classification system demonstrates high accuracy, strong generalization capability, and reliable prediction confidence. The model effectively classifies civic complaints into appropriate categories.These results validate the effectiveness of deep learning and transformer-based approaches in handling real-world text classification problems.

# References


[1] S. Sharma, A. Kar, and M. Gupta, "Building accountability in public service delivery through ICT: A case study of CPGRAMS," *Information Polity*, vol. 24, no. 2, pp. 205–220, 2019.

[2] D. Shukla and N. Singh, "Design and implementation of a smart grievance redressal system using mobile application," in *Proc. Int. Conf. Intelligent Communication and Computational Techniques (ICCT)*, 2019, pp. 253–258.

[3] M. Wani and R. R. Choudhary, "A review of text classification algorithms for complaint management system," *International Journal of Computer Applications*, vol. 179, no. 7, pp. 9–13, 2018.

[4] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[5] A. Chauhan, S. Chauhan, and R. Mathur, "Text classification in Hindi and Nepali using multilingual BERT," *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 11, no. 8, pp. 551–557, 2020.

[6] R. Sharma and A. Dey, "Detection of road surface conditions using deep convolutional neural networks," *Procedia Computer Science*, vol. 132, pp. 963–970, 2018.

[7] M. Jain and P. Kataria, "GIS-based civic complaint monitoring system using open source tools," in *Proc. Int. Conf. Geospatial Technologies (ICGT)*, 2021.

[8] P. Patil, M. Pandhare, and R. Jadhav, "Smart complaint management system using speech recognition," in *Proc. Int. Conf. on Innovative Mechanisms for Industry Applications (ICIMIA)*, 2020, pp. 334–338.

[9] S. Kulkarni and B. Bhosale, "Real-time municipal grievance system with notification mechanism," *International Journal of Engineering Research and Technology (IJERT)*, vol. 9, no. 5, pp. 164–168, 2020.